

Отчёт по лабораторной работе №13

Архитектура компьютеров и операционные системы

Степан Андреевич Гусев

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	8
4	Выполнение лабораторной работы	10
4.1	Первый скрипт	10
4.2	Второй скрипт	12
4.3	Третий скрипт	15
4.4	Четвёртый скрипт	15
5	Выводы	17
6	Ответы на контрольные вопросы	18
7	Список литературы	22

Список иллюстраций

4.1	Создание файла 1	10
4.2	Скрипт 1	11
4.3	Текст input.txt	12
4.4	Текст output.txt	12
4.5	Создание файла 2	13
4.6	Скрипт 2	13
4.7	Код С	14
4.8	Результат работы скрипта 2	14
4.9	Создание файла 3	15
4.10	Скрипт 3	15
4.11	Создание файла 4	16
4.12	Скрипт 4	16
4.13	Результат работы скрипта 4	16

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

- 1) Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами:
 - `-i inputfile` — прочитать данные из указанного файла;
 - `-o outputfile` — вывести данные в указанный файл;
 - `-р шаблон` — указать шаблон для поиска;
 - `-C` — различать большие и малые буквы;
 - `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-р`.
- 2) Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
- 3) Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до $\square$ (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
- 4) Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его

так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад.

3 Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек:

оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций;

C-оболочка (или csh) — надстройка на оболочке Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд;

оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна;

BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation).

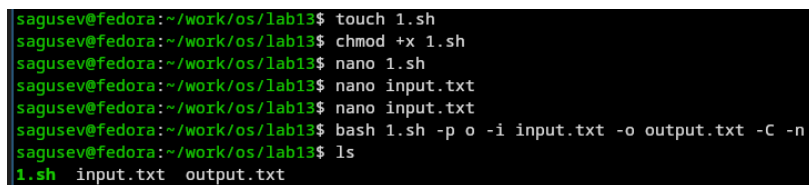
POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна. Рассмотрим основные элементы программирования в оболочке bash. В других оболочках большинство команд

будет совпадать с описанными ниже.

4 Выполнение лабораторной работы

4.1 Первый скрипт

Создал файл 1.sh, дал права на выполнение, ввёл код и запустил скрипт (рис. 4.1).



```
sagusev@fedora:~/work/os/lab13$ touch 1.sh
sagusev@fedora:~/work/os/lab13$ chmod +x 1.sh
sagusev@fedora:~/work/os/lab13$ nano 1.sh
sagusev@fedora:~/work/os/lab13$ nano input.txt
sagusev@fedora:~/work/os/lab13$ nano input.txt
sagusev@fedora:~/work/os/lab13$ bash 1.sh -p o -i input.txt -o output.txt -C -n
sagusev@fedora:~/work/os/lab13$ ls
1.sh  input.txt  output.txt
```

Рисунок 4.1: Создание файла 1

Текст скрипта 1.sh (рис. 4.2).

```
GNU nano 8.3
#!/bin/bash
while getopts i:o:p:C:n params
do
case $params in
    i) iflag=1; ival=$OPTARG;;
    o) oflag=1; oval=$OPTARG;;
    p) pflag=1; pval=$OPTARG;;
    C) cflag=1;;
    n) nflag=1;;
    *) echo Incorrect input $params;;
    esac
done
if ! test $cflag
then
    cf=-i
fi
if test $nflag
then
    nf=-n
fi
grep $cf $nf $pval $ival >> $oval
```

Рисунок 4.2: Скрипт 1

Текст input.txt (рис. 4.3).

```
nano input.txt
GNU nano 8.3
aaaaaaaaaaaaa o ppppppp
aaaaaaaaaoiiiiiii
o
aaaaaggg
sggdf
cmnxbnvxmcvxcv
werywruywieroowewerieweru
```

Рисунок 4.3: Текст input.txt

Результат работы скрипта (рис. 4.4).

```
nano output.txt
GNU nano 8.3
aaaaaaaaaaaaa o ppppppp
aaaaaaaaaoiiiiiii
o
werywruywieroowewerieweru
```

Рисунок 4.4: Текст output.txt

4.2 Второй скрипт

Создал файл 2.sh, дал права на выполнение, создал файл 2.c, дал права на выполнение (рис. 4.5).

```
sagusev@fedora:~/work/os/lab13$ touch 2.sh
sagusev@fedora:~/work/os/lab13$ chmod +x 2.sh
sagusev@fedora:~/work/os/lab13$ touch 2.c
sagusev@fedora:~/work/os/lab13$ chmod +x 2.c
sagusev@fedora:~/work/os/lab13$ nano 2.sh
sagusev@fedora:~/work/os/lab13$ nano 2.c
```

Рисунок 4.5: Создание файла 2

Текст скрипта 2.sh (рис. 4.6).

```
nano 2.sh
GNU nano 8.3
#!/bin/bash
gcc -o cprog 2.c
./cprog
case $? in
0) echo "Число равно нулю";;
1) echo "Число больше нуля";;
2) echo "Число меньше нуля";;
esac
```

Рисунок 4.6: Скрипт 2

Текст кода на Си (рис. 4.7).

```
nano 2.c
GNU nano 8.3
#include <stdlib.h>
#include <stdio.h>

int main() {
    int n;
    printf("Введите число: ");
    scanf("%d", &n);
    if (n>0) {
        exit(1);
    }
    else if (n==0) {
        exit(0);
    }
    else {
        exit(2);
    }
}
```

Рисунок 4.7: Код С

Запустил скрипт несколько раз, чтобы проверить все варианты (рис. 4.8).

```
sagusev@fedora:~/work/os/lab13$ bash 2.sh
Введите число: 1000
Число больше нуля
sagusev@fedora:~/work/os/lab13$ bash 2.sh
Введите число: 0
Число равно нулю
sagusev@fedora:~/work/os/lab13$ bash 2.sh
Введите число: -3
Число меньше нуля::
sagusev@fedora:~/work/os/lab13$
```

Рисунок 4.8: Результат работы скрипта 2

4.3 Третий скрипт

Создал файл 3.sh, дал права на выполнение, ввёл код и запустил скрипт (рис. 4.9).

```
sagusev@fedora:~/work/os/lab13$ touch 3.sh
sagusev@fedora:~/work/os/lab13$ chmod -x 3.sh
sagusev@fedora:~/work/os/lab13$ nano 3.sh
sagusev@fedora:~/work/os/lab13$ bash 3.sh 5
sagusev@fedora:~/work/os/lab13$ ls
1.sh 1.tmp 2.c 2.sh 2.tmp 2.c 3.sh 3.tmp 4.tmp 5.tmp cprog input.txt output.txt
sagusev@fedora:~/work/os/lab13$ bash 3.sh 5
sagusev@fedora:~/work/os/lab13$ ls
1.sh 2.c 2.sh 2.c 3.sh cprog input.txt output.txt
```

Рисунок 4.9: Создание файла 3

Текст скрипта 3.sh (рис. 4.10).

```
nano 3.sh
GNU nano 8.3
#!/bin/bash
for((i=1; i<=5; i++))
do
if test -f "$i".tmp
then rm "$i".tmp
else touch "$i.tmp"
fi
done
```

Рисунок 4.10: Скрипт 3

4.4 Четвёртый скрипт

Создал файл 4.sh, дал права на выполнение, открыл файл (рис. 4.11).

```
sagusev@fedora:~/work/os/lab13$ touch 4.sh
sagusev@fedora:~/work/os/lab13$ chmod -x 4.sh
sagusev@fedora:~/work/os/lab13$ nano 4.sh
```

Рисунок 4.11: Создание файла 4

Текст скрипта 4.sh (рис. 4.12).

```
nano 4.sh
GNU nano 8.3
#!/bin/bash
find $* -mtime 7 -mtime +0 -type f > files.txt
tar -cf files.tar -T files.txt
```

Рисунок 4.12: Скрипт 4

Запустил скрипт и проверил, что всё сработало корректно (рис. 4.13).

```
sagusev@fedora:~/work/os/lab13$ bash 4.sh /home/sagusev/
tar: Удаляется начальный '/' из имен объектов
tar: Удаляется начальные '/' из путей жестких ссылок
sagusev@fedora:~/work/os/lab13$ ls /home/sagusev/
#123.txt#  backup  git-extended  main.py  pipn-lock.yaml  Документы  Общедоступные
#124.txt#  bin  'lab07.sh#'  node_modules  text.txt  Загрузки  'Рабочий стол'
#347374#  Documents  lab07.sh  nothing  work  Изображения  Шаблоны
#3498tgher#  Downloads  LICENSE  package.json  Видео  Музыка
sagusev@fedora:~/work/os/lab13$ ls
1.sh  2.c  2.sh  2.c  3.sh  4.sh  cprog  files.tar  files.txt  input.txt  output.txt
sagusev@fedora:~/work/os/lab13$ cat files.t
files.tar  files.txt
sagusev@fedora:~/work/os/lab13$ cat files.txt
/home/sagusev/.mozilla/extensions/{ec8030f7-c20a-464f-9b0e-13a3a9e97384}/.fedora-langpack-install
/home/sagusev/.cache/mozilla/firefox/htd8aylw.default-release/cache2/entries/9C49283F3A31DC388B78EB426671A0E60544
0350
/home/sagusev/.cache/mozilla/firefox/htd8aylw.default-release/cache2/entries/63F48F4F7F1BC3195F5A8831F9794F3DBA2D
30E1
/home/sagusev/.cache/mozilla/firefox/htd8aylw.default-release/cache2/entries/C91C140F380A0AC5E92EC46CCCF38BD902D6
9D81
/home/sagusev/.cache/mozilla/firefox/htd8aylw.default-release/cache2/entries/B649E3A785D9DC9C4D631DA6C42458CFD282
34E0
/home/sagusev/.cache/mozilla/firefox/htd8aylw.default-release/cache2/entries/60DAFC5FDF8F854D2A25F04D522595E4F4FB
C3DF
/home/sagusev/.cache/mozilla/firefox/htd8aylw.default-release/cache2/entries/8392C4E5297289215EFEA88AE0AC36B7E318
```

Рисунок 4.13: Результат работы скрипта 4

5 Выводы

Я изучил основы программирования в оболочке ОС UNIX и научился писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

6 Ответы на контрольные вопросы

- 1) Осуществляет синтаксический анализ командной строки, выделяя флаги, и используется для объявления переменных. Синтаксис команды следующий: `getopts option-string variable`. Флаги – это опции командной строки, обычно помеченные знаком минус; Например, `-F` является флагом для команды `ls -F`. Иногда эти флаги имеют аргументы, связанные с ними. Программы интерпретируют эти флаги, соответствующим образом изменяя свое поведение. Строка опций `option-string` – это список возможных букв и чисел соответствующего флага. Если ожидается, что некоторый флаг будет сопровождаться некоторым аргументом, то за этой буквой должно следовать двоеточие. Соответствующей переменной присваивается буква данной опции. Если команда `getopts` может распознать аргумент, она возвращает истину. Принято включать `getopts` в цикл `while` и анализировать введенные данные с помощью оператора `case`. Предположим, необходимо распознать командную строку следующего формата: `testprog -infile_in.txt -ofile_out.doc -L -t -r` Вот как выглядит использование оператора `getopts` в этом случае:

```
while getopts o:i:Ltr optletter do
case optletter in
o) oflag = 1; oval =OPTARG;;
i) iflag=1; ival=$OPTARG;;
L) Lflag=1;;
t) tflag=1;;
r) rflag=1;;
*) echo Illegal option $optletter
esac
done
```

 Функция `getopts` включает две специальные переменные среды – `OPTARG` и `OPTIND`. Если ожидается дополнительное значение, то `OPTARG` устанавливается в значение этого аргумента (будет равна `file_in.txt` для опции `i` и `file_out.doc` для опции `o`). `OPTIND` является числовым индексом на

упомянутый аргумент. Функция `getopts` также понимает переменные типа массив, следовательно, можно использовать ее в функции не только для синтаксического анализа аргументов функций, но и для анализа введенных пользователем данных.

- 2) При перечислении имён файлов текущего каталога можно использовать следующие символы: `*` – соответствует произвольной, в том числе и пустой строке; `?` – соответствует любому одинарному символу; `[c1-c2]` – соответствует любому символу, лексикографически находящемуся между символами `c1` и `c2`. Например, `echo *` – выведет имена всех файлов текущего каталога, что представляет собой простейший аналог команды `ls`; `ls .c` – выведет все файлы с последними двумя символами, совпадающими с `.c`. `echo prog.?` – выведет все файлы, состоящие из пяти или шести символов, первыми пятью символами которых являются `prog.` `[a-z]` – соответствует произвольному имени файла в текущем каталоге, начинающемуся с любой строчной буквы латинского алфавита.
- 3) Часто бывает необходимо обеспечить проведение каких-либо действий циклически и управление дальнейшими действиями в зависимости от результатов проверки некоторого условия. Для решения подобных задач язык программирования `bash` предоставляет возможность использовать такие управляющие конструкции, как `for`, `case`, `if` и `while`. С точки зрения командного процессора эти управляющие конструкции являются обычными командами и могут использоваться как при создании командных файлов, так и при работе в интерактивном режиме. Команды, реализующие подобные конструкции, по сути, являются операторами языка программирования `bash`. Поэтому при описании языка программирования `bash` термин оператор будет использоваться наравне с термином команда. Команды ОС UNIX возвращают код завершения, значение которого может быть использовано для принятия решения о дальнейших действиях.

ях. Команда `test`, например, создана специально для использования в командных файлах. Единственная функция этой команды заключается в выработке кода завершения.

- 4) Два несложных способа позволяют вам прерывать циклы в оболочке `bash`. Команда `break` завершает выполнение цикла, а команда `continue` завершает данную итерацию блока операторов. Команда `break` полезна для завершения цикла `while` в ситуациях, когда условие перестаёт быть правильным. Команда `continue` используется в ситуациях, когда больше нет необходимости выполнять блок операторов, но вы можете захотеть продолжить проверять данный блок на других условных выражениях.
- 5) Следующие две команды ОС UNIX используются только совместно с управляющими конструкциями языка программирования `bash`: это команда `true`, которая всегда возвращает код завершения, равный нулю (т.е. истина), и команда `false`, которая всегда возвращает код завершения, не равный нулю (т.е. ложь).
- 6) Строка `if test -f mans/i.s, mans/i.s` и является ли этот файл обычным файлом. Если данный файл является каталогом, то команда вернет нулевое значение (ложь).
- 7) Выполнение оператора цикла `while` сводится к тому, что сначала выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, а затем, если последняя выполненная команда из этой последовательности команд возвращает нулевой код завершения (истина), выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `do`, после чего осуществляется безусловный переход на начало оператора цикла `while`. Выход из цикла будет осуществлён тогда, когда последняя выполненная команда из последовательности ко-

манд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, возвратит ненулевой код завершения (ложь). При замене в операторе цикла `while` служебного слова `while` на `until` условие, при выполнении которого осуществляется выход из цикла, меняется на противоположное. В остальном оператор цикла `while` и оператор цикла `until` идентичны.

7 Список литературы

1. https://esystem.rudn.ru/pluginfile.php/3097183/mod_resource/content/5/011-lab_shell_prog_2.pdf